DEBRE BERHAN UNIVERSTITY

College of Computing

Chapter 4: Exception

By Worku Muluye

August 13, 2026

Outline

- Debugging an Application
- Errors
- Exception Handling

Debugging an Application

- Debugging is the systematic process of identifying, analyzing, and fixing defects (bugs) within an application.
- In event-driven systems, debugging is often more challenging because bugs may only manifest under specific event sequences such as button clicks, mouse hovers, or form loads.
- The debugger provides richer insights into the application's dynamic state.
- Debugging in C# primarily involves using a debugger within an IDE/editor.
- Debugging allows developers to:
  - Observe and control program execution.
  - Inspect variable values and application state.
  - Trace the flow of execution.
  - Identify where a program's behavior deviates from expectations.

Debugging an Application: Debugging Workflow

- **Set a Breakpoint:** a marker that pauses program execution at a specific line of code.
  - Click in the margin beside the line number, or press F9.
  - A red dot appears to indicate the breakpoint.
- **Start Debugging:** Launch your application with the debugger attached.
  - Select Debug → Start Debugging, or Press F5.
  - The program runs until it reaches the first breakpoint.

Debugging an Application: Debugging Workflow

- **Inspect the Program State:** Once paused, examine the application's current state:
  - **Hover over variables:** View their values in data tips.
  - **Debugger windows**: Use Autos, Locals, and Watch to view variable values and expressions.
  - **Call Stack window**: Displays the sequence of method calls leading to the current point, helpful for understanding how a bug was triggered.

Debugging an Application

- **Control Execution Flow**: Use stepping commands to navigate code execution:
  - **Step Over (F10):** Execute the current line and move to the next (skip method details).
  - **Step Into (F11):** Dive into the method call to debug its implementation.
  - **Step Out (Shift + F11):** Finish the current method and return to the caller.
  - **Continue (F5):** Resume program execution until the next breakpoint.

Debugging an Application: Advanced Debugging Techniques

- **Conditional Breakpoints:** Pause execution only when a specified condition is met.
  - Useful for bugs that appear under certain conditions (e.g., within loops).
  - Right-click a breakpoint → Conditions → Enter the condition.
- **Tracepoints (Logpoints):** Breakpoints that log messages to the Output window without stopping execution.
  - Right-click a breakpoint → Actions → Enter a custom log message.
- **Watch Window:** Monitor specific variables or expressions across different scopes, even when they go out of the current context.

Debugging an Application: Advanced Debugging Techniques

- **Attach to a Process:** Debug an already running application (e.g., web server or background service).
  - Select Debug → Attach to Process, then choose the target process.
- **Exception Handling:** Configure the debugger to break whenever a specific exception is thrown, even if caught.
  - This helps locate the root cause before the exception is handled or ignored.
- Debug : Use the Debug class from the System.Diagnostics namespace for in-code debugging support. Examples:
  - Debug.WriteLine("Variable value: " + value);
  - Debug.Assert(value > 0, "Value must be positive");

Errors

- Errors are unintended deviations from expected behavior that prevent correct program execution.
- In C#, errors are broadly categorized as:
- **Compile-Time Errors:**
  - Errors are detected by the compiler **before execution.**
  - Easy to fix, as they are flagged during the build process.
  - Examples, syntax mistakes, type mismatches, and missing references.

Errors

- **Runtime Errors:**
  - It occurs while the program is executing.
  - Commonly arise due to invalid operations such as division by zero, invalid casting, and accessing null objects.
  - Harder to detect because they depend on dynamic user inputs or environmental conditions.
- **Logical Errors:**
  - The program compiles and runs but produces incorrect output. for example, implementing an incorrect algorithm logic.
  - Debugging logical errors often requires test-driven development (TDD) and thorough analysis.

Errors

- **Error Sources in Event-Driven Applications:**
  - Unhandled user input, such as invalid formats, null data.
  - Resource-related issues, such as file not found, database connection loss.
  - Asynchronous operations that fail silently.
  - Event propagation order, where handlers trigger in unexpected sequences.

Exception Handling

- An exception is a special type of runtime error represented by an object derived from the base class System.Exception.
- Exceptions in the application must be handled to prevent crashing of the program and unexpected results, log exceptions, and continue with other functionalities.
- Exception Handling is a mechanism to handle runtime errors in a controlled manner.
- It prevents abnormal program termination and provides user-friendly error messages.
- Exception handling provides a structured mechanism to detect, respond to, and recover from such runtime anomalies without crashing the application.
- In C#, exceptions are objects derived from the Exception base class.
- C# provides built-in support to handle the exception using **try, catch,** and **finally** blocks.

Exception Handling

- An Exception is a class in C# that represents runtime errors.
- It is responsible for the abnormal termination of a program when an error occurs.
- Exception classes examples:
  - DivideByZeroException
  - NullReferenceException
  - IndexOutOfRangeException
  - FileNotFoundException
  - FormatException
- Note: Runtime errors are not exceptions themselves; exceptions are objects that describe and handle those errors.

Exception Handling

- **Exception in C#:**
  - When an exception occurs, the runtime (Common Language Runtime) searches the call stack for a matching catch block.
  - If no handler is found, the program terminates with an unhandled exception error.
- **Role of CLR in Exception Handling**
  - When a runtime error occurs, the CLR identifies the error type.
  - It creates an object of the corresponding Exception class.
  - The object is thrown (using the throw keyword).
  - The program terminates abnormally unless the exception is handled.

Exception Handling

- Exception Handling is important to:
  - Prevent abnormal program termination.
  - Provide meaningful messages to users.
  - Perform corrective actions when possible.
  - Maintain application stability and reliability.

Exception Handling

```
class Program
{
    static void Main(string[] args)
    {
        int a = 20;
        int b = 0;
        int c;
        Console.WriteLine
        Console.WriteLine
        c = a / b;
        Console.WriteLine
        Console.ReadKey()
    }
}
```

Exception Unhandled

System.DivideByZeroException: 'Attempted to divide by zero.'

View Details | Copy Details

Exception Settings

The CLR terminates the program by throwing a DivideByZeroException due to dividing an integer by zero.

Exception Handling: Approaches in C#

- **Logical Implementation:**
  - Handle predictable errors using logical conditions (if-else). Example:

```
if (Num2 == 0)
    Console.WriteLine("Second number should not be zero.");
else
    Result = Num1 / Num2;
```

  - It is simply, predictable errors. Fails to handle unexpected exceptions.
- **Try-Catch Implementation:**
  - Used when logical handling is insufficient.

Exception Handling: Try-Catch-Finally Components

- **Try Block:** Contains code that may generate exceptions.
- **Catch Block:** Catches and handles exceptions thrown from the try block.
  - Can have specific or generic exception handling.
- **Finally Block:** Executes regardless of whether an exception occurs.
  - The finally block in C# always executes, whether or not an exception occurs in the try block.
  - It is typically used for resource cleanup (closing files, releasing connections, freeing memory, etc.), closing database connections, and resetting variables or states.
  - To ensure that cleanup code executes even when an exception occurs.

Exception Handling: Try-Catch-Finally Components

```
try{
    //code that might throw an exception
}
catch (FormatException ex) {
    //handle specific exception
}
catch (Exception ex) {
//handle any other exception
}
finally {
   //always executes (cleanup: close files, release resources)
}
```

```
void SomeMethod()
{
     //Normal Statements
     try
     {
          //Exception causing and its related statement which
          //we don't want to execute when exception occurred
     }
    catch (SomeException1 one)
    {
          //Statements which required to execute when SomeException1
          //Occcurred in the program
    }
    catch (SomeException2 two)
    {
          //Statements which required to execute when SomeException2
          //Occcurred in the program
    }
    catch (Exception generic)
    {
          //This is generic exception handling block. If non of the above
          //Exception catch block handles the exception, then this catch
          //block is going to be execute
    }
   //Normal Statements
}
```

Exception Handling

- **try block:** Any suspected code that may raise exceptions should be put inside a try{ } block.
- During the execution, if an exception occurs, the flow of the control jumps to the first matching catch block.
- **catch block:** The catch block is an exception handler block where you can perform some action, such as logging and auditing an exception.
- The catch block takes a parameter of an exception type, using which you can get the details of an exception.

Exception Handling: Try-Catch Structure

- **try–catch:**
  - Handles exceptions and prevents abnormal termination.
- **try–catch–finally**
  - Handles exceptions and ensures cleanup code executes.
- **try–finally**
  - Executes cleanup code without handling the exception.
  - Abnormal termination still occurs, but resources are released.

Exception Handling: Multiple Catch Blocks

- In C#, a single try block can be followed by multiple catch blocks to handle different types of exceptions.
- This approach allows developers to manage specific exception types separately and provide customized error handling for each case.
- We use multiple catch blocks when:
  - We need to display messages specific to each exception.
  - We need to execute different logic based on the type of exception thrown.
- Rules for Multiple Catch Blocks
  - Only one catch block executes for a given exception.
  - The first matching catch block will handle the exception.
  - Catch blocks must be ordered from most specific to most general (i.e., child exceptions first, then parent).

Exception Handling

- **finally block:** The finally block will always be executed whether an exception is raised or not.
- Usually, a finally block should be used to release resources, e.g., to close any stream or file objects that were opened in the try block.
- A try block must be followed by a catch or finally block, or both blocks.
- The try block without a catch or finally block will give a compile-time error.

Exception Handling: Example

```
try {
int a = int.Parse(textBox1.Text);
int b = int.Parse(textBox2.Text);
int c = a / b;
textBox3.Text = c.ToString();
}
catch {
MessageBox.Show("Error occurred.");
}
finally {
MessageBox.Show("Re-try with a different number.");
}
```

Exception Filters

- We can use multiple catch blocks with different exception type parameters; this is called exception filters.
- Exception filters are useful when you want to handle different types of exceptions in different ways.
- Multiple catch blocks with the same exception type are not allowed.
- A catch block with the base Exception type must be the last block.

Exception Classes in C#

- C# exceptions are represented by classes.
- The exception classes in C# are mainly directly or indirectly derived from the System.Exception class.
- Some of the exception classes are derived from the System.Exception classes are the System.ApplicationException and System.SystemException classes.
- The System.ApplicationException class supports exceptions generated by application programs.
- Hence, the exceptions defined by the programmers should derive from this class.
- The System.SystemException class is the base class for all predefined system exceptions.

Exception Classes in C#

Predefined exception classes derived from the Sytem.SystemException class:

- **Arithmetic Exceptions**
  - DivideByZeroException: occurred when dividing by zero in integer operations.
  - OverflowException: occurred when an arithmetic operation exceeds the range of the data type.
  - ArithmeticException: Base class for arithmetic-related exceptions.
- **Invalid Operation Exceptions**
  - InvalidOperationException: occurred when a method call is invalid in the current state of an object.
  - NotSupportedException: occurred when an invoked method or operation is not supported.
  - ObjectDisposedException: occurred when operations are performed on a disposed object.

Exception Classes in C#

Predefined exception classes derived from the Sytem.SystemException class:

- **Indexing and Access Exceptions:**
  - IndexOutOfRangeException: Raised when accessing an array element outside its bounds.
  - KeyNotFoundException: Raised when trying to access a dictionary element with a missing key.
  - NullReferenceException: Raised when trying to access a member of a null object.
- **Type and Casting Exceptions:**
  - InvalidCastException: Raised when a cast to an incompatible type is attempted.
  - ArrayTypeMismatchException: Raised when storing the wrong type of object in an array.
  - TypeLoadException: Raised when the runtime fails to load a type..

Exception Classes in C#

Predefined exception classes derived from the Sytem.SystemException class:

- **I/O and File Handling Exceptions:**
  - IOException: Base class for input/output errors.
  - FileNotFoundException: occurred when a file is not found.
  - DirectoryNotFoundException: occurred when a directory path is invalid.
  - EndOfStreamException: occurred when reading past the end of a stream.
- **Security and Access Exceptions:**
  - UnauthorizedAccessException: occurred when the program tries to access a resource without permission.
  - SecurityException: occurred when a security error is detected.

Example

```
try{
    int a = int.Parse(textBox1.Text);
    int b = int.Parse(textBox2.Text);
    int c = a / b;
    textBox3.Text = c.ToString();
}
catch (DivideByZeroException) {
    MessageBox.Show("Cannot divide by zero. Please try again.");
}
catch (FormatException) {
    MessageBox.Show("Input must be a valid number.");
}
catch (Exception ex){
    MessageBox.Show($" Unexpected error: {ex.Message}"); }
```

Invalid Catch Block

- A parameter-less catch block and a catch block with the Exception parameter are not allowed in the same try-catch statements, because they both do the same thing. E.g.

```
try {
//code that may raise an exception
}
catch { //can't have both catch and catch(Exception ex)
MessageBox.Show("Error occurred.");
}
catch(Exception ex) { //can't have both catch and catch(Exception ex)
MessageBox.Show("Exception occurred"); }
```

Why invalid?

- The first catch (parameterless) already catches every exception.
- The second catch (Exception ex) also catches every exception.
- So, the compiler says: Duplicate catch blocks for the same exception type are not allowed.

Invalid Catch Block: Solution

- **Parameter-less catch:**

```
try
{
    int x = int.Parse("abc"); // FormatException
}
catch
{
    MessageBox.Show("Error occurred.");
}
```

- **Catch with Exception ex:**

```
try
{
    int x = int.Parse("abc"); // FormatException
}
catch (Exception ex)
{
    MessageBox.Show($"Error: {ex.Message}");
}
```

Nested try-catch

- C# allows nested try-catch blocks.
- A nested try-catch is a try-catch block placed inside another try or catch block.
- When using nested try-catch blocks, an exception will be caught in the first matching catch block that follows the try block where an exception occurred.
- It allows handling different exceptions at different levels.
- Useful when one operation might throw multiple types of exceptions that need different handling strategies.

Nested try-catch: Example

```
try {
    int a = int.Parse("100");
    int b = int.Parse("0");
    try  {
        int c = a / b;  // Inner risky operation
        Console.WriteLine($"Result: {c}");
    }
    catch (DivideByZeroException ex) {
        Console.WriteLine("Inner Catch: Cannot divide by zero.");
    }
    Console.WriteLine("Outer Try continues...");
}
catch (FormatException ex)
{
    Console.WriteLine("Outer Catch: Invalid number format.");
}
catch (Exception ex) {
    Console.WriteLine($"Outer Catch: General error - {ex.Message}");
}
```

Creating User-Defined Exceptions(Custom Exceptions)

- C# provides a rich hierarchy of exception classes under the System.Exception namespace.
- Sometimes, predefined exceptions like DivideByZeroException, FormatException, etc., are not enough for specific business rules.
- In such cases, we create custom (user-defined) exceptions.
- **System Exceptions**: are thrown automatically by the CLR due to runtime errors. E.g. DivideByZeroException, FormatException, NullReferenceException
- **Application Exceptions:** are thrown explicitly by developers for custom business rules: OddNumberException (user-defined)ns.
- **Why Create Custom Exceptions?**
  - When no existing .NET exception fits the scenario.
  - To enforce business logic validation.
  - To provide clear, descriptive error messages.
  - To separate system-level errors from application-level errors.

Creating User-Defined Exceptions

- A user-defined (custom) exception is a class that you create to represent a specific error condition in your application.
- It allows programmers to programmatically distinguish between different error types.
- Improves error handling clarity and overall code quality.
- Especially useful in large-scale projects where generic exceptions are not enough.
- **Inheritance:**
  - Custom exceptions are derived from **ApplicationException.**
  - This separates them from system exceptions (SystemException), which are CLR-defined.
- **Naming Convention**: Name your custom exception ending with Exception.
- Advantages
  - Clearer code: Each exception has a meaningful name.
  - Easier to handle specific errors in try-catch blocks.
  - Improves maintainability and readability of large-scale applications.

Creating User-Defined Exceptions: Example

```
public class MyCustomException : System.Exception
{
    public MyCustomException() : base() { }
    public MyCustomException(string message) : base(message) { }
}

try {
    int a = int.Parse(textBox1.Text);
    int b = int.Parse(textBox2.Text);
    int c = a / b;
    textBox3.Text = c.ToString();
}
catch (Exception ex)
{
    throw new MyCustomException("You cannot divide a number by zero");
}
```

Thank You!

?